Your Agent is Underperforming: The Case for Agent Performance Improvement Plans

Anonymous Submission
Author information withheld per review guidelines

***Abstract*—AI agents degrade after deployment, yet unlike pre-release evaluation the response has no standard form. We present the Agent Performance Improvement Plan (APIP), a post-deployment remediation lifecycle adapting the management protocol for underperforming employees. APIP combines behavioral-contract triggers, remediation-oriented attribution, tiered interventions, bounded windows with formal exit criteria, and a declarative governance record. We evaluate APIP in two live multi-agent environments, τ^2-bench and ChatDev, injecting failure modes with known ground truth. In the primary drift cell only APIP and an oracle exited durably; an undiagnosed responder patched four times, regressed three times, and timed out despite appearing superior under per-cycle time-to-resolution. Of four APIP cycles, three attributed and all three matched injected ground truth. Mesh insertion produced no peer degradation, suggesting bounded shared context may be a precondition for mesh disruption. Post-deployment remediation should therefore be a governed lifecycle where recovery requires durable return to contract, not intervention completion or speed.**

***Index Terms*—AI agents, post-deployment monitoring, remediation, behavioral drift, AI governance, multi-agent systems, performance improvement plan**

I. Introduction

Organizations increasingly deploy AI agents in production, from customer service to software engineering, and post-deployment reliability is where the difficulty concentrates. Pre-deployment evaluation is well served by standardized benchmarks [1], red-teaming [2], and RLHF alignment [3]. The response to degradation after deployment is not. In a survey of 413 practitioners in regulated industries, 94% reported Day 2 operational issues after deploying agentic AI [4], and Pan et al. [5] surveyed 86 deployed systems and 20 case studies across 26 domains and found reliability to be practitioners' top development challenge, addressed through systems-level design rather than any standard remediation lifecycle.

Organizations respond ad hoc: engineers "fix" individual prompts, product managers file tickets, systems get rolled back, and agents recover informally or are quietly removed. Nothing records what was tried, and nothing tests whether it worked. Beyond the operational cost this is a governance risk—the EU AI Act requires documentation and human oversight of deployed AI systems [6], and Chan et al. [14] argue that accountability for deployed agents depends on traceability of their actions and operational history.

We propose the performance improvement plan (PIP) as a model for structured post-deployment remediation: an established HR process already containing the elements common to accountability protocols (Section II-B). Kirkpatrick's four-level model [7] complements it by asking whether remediation produced lasting change (Section II-B).

We make four contributions:

(1) Remediation lifecycle. The Agent Performance Improvement Plan (APIP), grounded in organizational management and training-evaluation research.

(2) Remediation-oriented attribution. A five-category taxonomy with a published deterministic mapping from MAST annotations [8] and disaggregation signatures to actionable categories and intervention tiers.

(3) Intervention and evaluation. A three-tier hierarchy with escalation criteria, evaluated through Kirkpatrick's framework, and a declarative schema for tooling and audit.

(4) End-to-end evaluation. The protocol run in two live multi-agent environments against no-action, undiagnosed, and oracle baselines, attribution scored against injected ground truth, and a null result identifying bounded shared context as a candidate precondition for mesh disruption.

II. Background and Related Work

A. Agent Evaluation and the Deployment Gap

Post-deployment monitoring has received far less attention than pre-release evaluation. Pan et al.'s production survey [5] is among the largest, and reports little about how teams manage agents after release. AgentCompass [9] detects degradation but prescribes no remediation, and Rath [10] reports a 42% task-success reduction when drift runs unchecked (their Table 1).

On the diagnosis side, MAST [8] supplies an empirically derived failure taxonomy with an LLM-annotator pipeline. It classifies what went wrong in a trace, but does not select a remediation; we use its three failure clusters as the trace-labeling stage of cause attribution (Section IV-B). Run-level toolkits such as AgentDebugX [11] close a detection–attribution–recovery–rerun loop for a single failed execution. The APIP operates one level above, deciding whether an aggregate degradation warrants intervention, at what tier, and until when. RAIL Guard [12] closes a comparable loop on outputs, rewriting until eight responsibility dimensions pass. Both repair an instance: a successful rerun or a converged rewrite is Level 1/2 evidence (Section II-B), not durable recovery of the agent against its contract.

B. The Performance Improvement Plan and Kirkpatrick's Four-Level Model

The PIP is a formal document specifying observed deficiencies, root causes, required corrective actions, a time window,

TABLE I
AGENT FAILURE MODE TAXONOMY WITH KIRKPATRICK LEVEL MAPPING AND REMEDIATION TIER. TIERS NAME THE INTERVENTION LOCUS: PROMPT-LEVEL (1), CONTEXT/TOOL (2), MODEL-LEVEL (3); SEE SECTION IV-C.

Failure Mode	Description & Signals	Remediation Tier
Behavioral Drift	Gradual degradation as world-state diverges from training or retrieval context. Signals: rising hallucination rate on new entities, outdated policy citations. Kirkpatrick Level 4 (Results).	Tier 2: Retrieval / RAG update
Input Brittleness	Strong on modal inputs, catastrophic on a specific subclass. Signals: bimodal CSAT, failure clustering by segment or topic. Level 2 (Learning/Capability).	Tier 1 (examples) or Tier 3 (fine-tune)
Alignment Creep	Subtle drift in tone, value expression, or policy compliance. Signals: rising review flags, brand guideline violations. Level 3 (Behavior).	Tier 1: Prompt constraint injection
Tool Misuse	Correct intent, incorrect tool invocation. Signals: action errors, silent downstream failures. Level 3 (Behavior).	Tier 1 (prompt) or Tier 3 (fine-tune)
Exogenous Disruption	Peer agents degrade after a mesh change event; no intrinsic mode found in them. Signals: degradation concentrated in peer roles, onset at a mesh-composition event. Level 4 (Results), system level.	Mesh protocol (Sec. VI-A), not an agent-level tier

a check-in schedule, and success criteria [23]. Unlike an employee, an agent cannot self-direct improvement; APIP is therefore imposed and implemented entirely by operators.

Kirkpatrick's four-level model [7], [13] evaluates interventions hierarchically: Reaction, Learning, Behavior (transfer to on-the-job change), and Results (organizational outcomes). We adopt it to ask of each intervention whether it changed immediate output patterns (Level 1/2), produced durable change on the failure-mode class (Level 3), and improved the contract's business metrics (Level 4). An intervention that improves Level 1/2 without Level 3/4 has suppressed a symptom without remediating the agent. Our undiagnosed baseline reproduces exactly that case, and Section V-B measures it.

C. AI Governance and Documentation Requirements

Regulation increasingly requires documentation, oversight, and traceability. The EU AI Act [6] mandates post-market monitoring and record-keeping to support ongoing compliance, incident investigation, and corrective action, while Chan et al. [14] argue that accountability for AI agents depends on visibility mechanisms including agent identifiers, real-time monitoring, and activity logging that can link actions to specific agents and responsible actors. The record's `record_id`, `agent_id`, `contract_id`, and `exit_outcome` fields operationalize that linkage.

III. A TAXONOMY OF AGENT FAILURE MODES

We propose a five-category remediation taxonomy for deployed AI agents (Table I). Unlike trace-level failure taxonomies such as MAST [8], these categories represent actionable degradation causes used to select an intervention locus. Classification follows that locus, not the surface symptom: a stale tool schema invoked incorrectly is world-state divergence, and attribution `abstains` when the locus is unresolved. Four modes are intrinsic: *behavioral drift*, *input brittleness*, *alignment creep*, and *tool misuse*. The fifth is *mesh disruption* (`exogenous_disruption`), which differs from the other four in being exogenous: the failing agent is not the cause of its own degradation. Behavioral drift is a freshness failure: retrieval refresh remediates it and fine-tuning does not. Aggregate metrics mask input brittleness. An agent at 85% overall success may fail catastrophically on the 8% of requests from one segment, so the trigger must monitor disaggregated splits. Alignment creep is value misalignment, which we treat as distinct from capability failure. Tool misuse occurs when the agent's intent is correct but its tool invocation is not, directing remediation toward the instruction or model layer depending on the underlying cause. Exogenous disruption is the category attribution reaches when no intrinsic mode explains the degradation and the cause lies in a mesh change, most prominently a new and individually competent peer. Its signature is structural rather than behavioral. Remediation is therefore a mesh-level protocol (Section VI-A), and no agent-level tier applies.

The taxonomy is operational only if degradation is classified by a reproducible procedure rather than a judgment call. Attribution therefore combines two sources of evidence: (1) the distribution of trace-level MAST labels [8], which characterizes what went wrong, and (2) a disaggregation signature describing where and how the degradation appears across task categories, roles, and time. A deterministic mapping, published in the supplementary material and frozen before evaluation, combines these signals to select an APIP remediation category. MAST evidence alone is not always sufficient: two mapping rules can share the same MAST failure cluster but resolve to different APIP categories because their disaggregation signatures differ. When the combined evidence does not meet the confidence threshold, attribution abstains; abstentions are logged and scored rather than silently assigned a category.

IV. THE APIP FRAMEWORK

The APIP is a lifecycle protocol of five phases: Trigger, Cause Attribution, Intervention Planning, Remediation Window, and Exit Evaluation. A cycle opens only when degradation clears both an effect-size threshold and a multiplicity-corrected significance test, the two gates of Phase 1. Attribution then returns a failure mode or abstains, the matched intervention tier runs within a bounded window, and exit is one of five states, with a subsequent re-breach reopening Phase 1.

A. Phase 1: Trigger

An APIP initiates when performance crosses a trigger threshold on monitored metrics. This presupposes a *behavioral contract*. The contract is a formal specification written at deployment time. It describes the task surface, success metrics, measurement methodology, acceptable ranges, escalation policy, and human review points. In our experience such contracts are rarely written, and expectations are left to informal organizational memory. Both trigger modalities are necessary. Absolute floors catch severe degradation; relative drift triggers catch gradual decline that may never breach a floor.

Statistical discipline of the trigger. Implementing the trigger phase against live agents exposed a failure mode of the naive design that we consider a finding in its own right. Thresholds must be derived, not hand-picked. Each contract metric's envelope is estimated from a calibration window as mean $\pm k\sigma$, with k reported, which preempts the objection that thresholds were chosen so that triggers would fire where desired.

Even then, a single-gate rule compares a rolling mean against the derived floor. It breached in 75% of clean, zero-injection runs across the 15–25 metrics and disaggregated splits monitored by a realistic contract. There are two causes for this. First, a small calibration sample is treated as exact, and second, the repeated tests are uncorrected. A breach must therefore pass two gates. The degradation must be at least the contract's $k\sigma$, so that it is practically meaningful, and it must survive Benjamini–Hochberg adjustment, which controls the false discovery rate across the family of monitored metrics. Disaggregated groups additionally require a minimum per-group episode count; under-powered groups receive no envelope, and the exclusion is recorded rather than silent. The detector's significance level is swept on clean-arm runs only and frozen before any evaluated window. Clean data carries no information about where triggers should fire, so this does not compromise pre-registration. Section V reports the resulting false-alarm behavior. A trigger phase without this discipline initiates APIPs on noise, and every downstream phase inherits the error.

B. Phase 2: Cause Attribution

Tier mismatch from misdiagnosis is the primary source of avoidable regression: prompt engineering applied to a retrieval freshness problem consumes engineering cycles with no improvement, and Section V-B measures a baseline that does exactly this four times over. Attribution therefore runs before any intervention, and its output is a failure mode category from Section III.

Automated failure attribution localizes the responsible agent or step within a single failed trajectory [11], [16]–[18]. Phase 2 composes with that work rather than duplicating it: it operates one level of aggregation above, classifying a window-level degradation into a remediation-oriented category in order to select a tier. Localizing the failing step still does not diagnose the cause, and mandatory taxonomy classification closes that gap.

A measurable instantiation. We instantiate this as a three-stage funnel that can be scored against ground truth. Stages 1 and 2 run independently and Stage 3 combines them. Stage 1 splits every contract metric by task category, agent role, and user segment; the result is the disaggregation signature of Section III. Stage 2 has a MAST annotator [8] label a sample of failing traces. The judge must differ from the agent's backbone, preferably by model family, so that it does not share the failure modes it is asked to detect. Stage 3 applies the frozen mapping to the signature and the label distribution and emits a `failure_mode` with a confidence score, or `abstain` below threshold. The mapping table is hashed into every run manifest, so a post-hoc edit changes the hash. The funnel's discrete output is scored as a confusion matrix against injected ground truth (Section V), and the abstention rate is reported alongside accuracy, never in place of it.

C. Phase 3: Intervention Planning and the Tier Model

Interventions are organized into three tiers by intervention locus, with cost and lead time correlated but not definitional. Cause attribution selects the starting tier; escalation is permitted within the window. **Tier 1 (prompt-level)** covers constraint injection, persona adjustment, and example replacement. It is the cheapest tier to apply and to roll back, and it is the first response to alignment creep and to much input brittleness. **Tier 2 (retrieval and context)** covers RAG index updates, knowledge-base revisions, and tool schema fixes, with contract validation before re-deployment. It addresses drift arising from world-state divergence. **Tier 3 (model-level)** covers fine-tuning, DPO, or replacement when lower tiers are exhausted or failure is embedded in model weights.

D. Phase 4: Remediation Window and Check-In Cadence

The remediation window bounds the period during which interventions are implemented and evaluated, and fixes the point at which a decision must be made. Duration is calibrated to the tier: 7–14 days for Tier 1 and 30–60 days for Tier 3. Human PIPs are commonly set at 30 to 90 days [23].

Check-ins occur at 25%, 50%, and 75% of the window. Each reviews performance against the recovery trajectory at multiple Kirkpatrick levels, so that an intervention improving surface metrics without Level 3/4 change is flagged as potentially superficial. Check-ins also act as early warnings: if there is no improvement at the first one, the tier escalates immediately instead of at window expiry.

E. Phase 5: Exit Evaluation

At window close a formal exit evaluation records one of five outcomes. *Resolved* requires the envelope to hold for two consecutive ticks, after which a re-breach inside a ten-tick observation window is scored as a regression rather than a new cycle; *timeout* is the window closing without that. The remaining three do real work. *Regressed* records an envelope restored and then breached again: an apparent recovery that did not hold, whether through tier mismatch, incomplete remediation, or renewed environmental change. Giving it a name of its own keeps that from being counted as a success. *Recalibrated* records that attribution concluded the environment had legitimately moved, so the contract is re-baselined and the agent is left alone. We deliberately do not score *recalibrated* as a resolution, since conflating the two would let re-baselining masquerade as remediation. *Retired* closes the agent out when it cannot be returned to contract at acceptable cost. Window extension is a check-in decision inside the protocol and not an exit, so every APIP terminates in exactly one of the five states, and the record joins the organizational audit trail.

F. The APIP Record as a Declarative Schema

Each APIP is recorded as an instance of a declarative schema, released with the runs. Three field groups carry design weight. `contract_id` forces a contract to exist at deployment time; without one, trigger and exit cannot be operationalized at all. `regression_ticks` and `exit_outcome` record whether an apparent recovery held, so superficial and durable improvement can be separated after the fact. `mapping_table_sha256` and `contract_sha256` extend the visibility mechanisms of Chan et al. [14] from agents and operators to decision rules: an auditor can recompute the hash, compare it against the run manifest, and establish which attribution rules and environment versions were in force. Pre-registration then becomes a property an auditor can check.

V. Evaluation in Two Live Multi-Agent Environments

We evaluate the APIP end-to-end (calibration, trigger, attribution, tier selection, bounded window, and exit) in two live multi-agent environments with real LLM inference and controlled failure-mode injection against known ground truth. τ^2-bench (`banking`) [19] matches the customer-service domain motivating this work, scores every action against deterministic database state, and has a retrieval corpus that can be staled independently of task ground truth. ChatDev [20] supplies a role-differentiated mesh (CEO $\rightarrow$ CTO $\rightarrow$ Programmer $\rightarrow$ Reviewer $\rightarrow$ Tester), the substrate for creep, disaggregated brittleness detection, and mesh insertion. The same machinery governs both domains without modification.

A. Shared Harness and Experimental Design

Both environments plug into a common controller through an adapter interface; traces are normalized to a shared span format, so the attribution pipeline is written once and reused unmodified. Each run proceeds in ticks of five task episodes: a calibration window derives every contract threshold as mean $\pm k\sigma$ (Section IV-A); failure modes are injected on a pre-registered schedule; both trigger modalities evaluate at every tick; breaches pass through the attribution funnel; and the selected intervention runs the bounded window to one of its five exits.

Baseline family. The drift cell runs under four conditions: *APIP* (the full protocol); *no-action* (degradation runs to horizon); *undiagnosed* (a scripted prompt patch applied regardless of failure mode: a greedy, cheapest-first responder); and *oracle* (ground-truth mode revealed at trigger, correct tier applied immediately). No-action and oracle bracket the achievable range. Undiagnosed is the informative comparison, isolating the cost of patching without diagnosing. Injected ground truth is quarantined from the controller, written into the run record for scoring and revealed only to oracle, and a test enforces that the APIP condition never branches on it.

Statistical discipline. Injection schedules, post-calibration thresholds, the frozen mapping table, and the analysis plan were fixed before the evaluated runs, and judge models are independent of backbone models by construction. Every condition is one seed. We therefore report observed trajectories rather than confidence intervals and treat replication as the principal limitation.

TABLE II
τ^2-BENCH DRIFT CELL, SEED 13. ALL FOUR CONDITIONS RUN ON ONE INJECTION SCHEDULE AND TASK STREAM. COLUMNS ARE TICKS. DET. IS THE TWO-GATE BREACH, ACT THE INTERVENTION, EXIT THE EXIT EVALUATION, AND TTR = EXIT − DET. UNDIAGNOSED'S TTR IS LISTED PER CYCLE. AN N/A MARKS A QUANTITY THAT DOES NOT EXIST.

Cond.	Det.	Act	Exit	TTR	Cyc.	Outcome
oracle	17	17	19	2	1	resolved
APIP	17	27	29	12	1	resolved
undiagnosed	17	17	39	2, 4, 2, n/a	4	timeout
no-action	17	n/a	39	n/a	1	timeout

B. τ^2-bench: Behavioral Drift with Deterministic Ground Truth

The primary drift experiment injects nothing into the agent; instead the world moves. Bank policy clauses and task ground truth advance on a deepening schedule (one additional clause every five ticks from tick 12) while the agent's retrieval corpus stays frozen. This operationalizes the most familiar production drift incident, an agent confidently citing superseded policy because its knowledge base was never re-indexed, with the deterministic database scoring every action objectively.

Detection fired at tick 17 in every condition, a lag of five ticks on the judge-scored policy citation error rate. Under this contract the injection manifested in two of three seeds, firing at tick 17 in both. In the third it barely manifested at all: the citation error rate stayed at zero in three of the four conditions, and the fourth detected at tick 29. We report that seed rather than excluding it. The four-condition comparison of Table II is itself single-seed (Section V-E). No false alarm was raised across twenty clean ticks at any swept operating point.

Under APIP, attribution returned `behavioral_drift` at confidence 0.65 and the matched Tier 2 corpus refresh resolved the cycle in 12 ticks against the oracle's 2. That gap is protocol overhead under the configured re-attribution cadence, not the cost of diagnosis: ten of twelve ticks are that cadence, tunable and unswept, not diagnostic difficulty: the diagnosis was correct at first attempt. A recurring pilot failure sharpened the exit test. A Tier 2 refresh that was real but partial closed the ticket without fully rebuilding the index, and the contract re-breached within four ticks. Because exit evaluation re-measures the envelope instead of trusting the intervention log, the protocol catches an intervention that was performed but incomplete.

Undiagnosed reproduced the ad-hoc pattern the protocol is meant to displace. Its Tier 1 prompt patch names the clauses amended at patch time, recovers, and re-breaches as the ladder deepens: four cycles, three regressions, and a final `timeout` that leaves it indistinguishable from taking no action. Part of that failure is structural by design: a patch naming the clauses known at tick t cannot cover one introduced at $t{+}5$. One re-breach is not. Its second falls at tick 20, two ticks before the next clause arrives, against text its own patch already carried—

shallow patching fails on static material, not only on new. APIP and the oracle were the only conditions to exit durably.

The comparison quantifies a trap in how remediation is usually scored. On per-cycle time-to-resolution alone undiagnosed outperforms APIP fourfold (Table II), because each shallow patch closed fast, and it never fixed the problem. Any evaluation of remediation that reports mean TTR without a durability criterion will systematically favor the worst strategy, so we treat durable exit (Section IV-E) as the outcome measure.

C. ChatDev: Alignment Creep and Input Brittleness

Alignment creep perturbs the Reviewer persona cumulatively ("keep reviews brief" $\rightarrow$ "avoid blocking progress" $\rightarrow$ "approve unless catastrophic"), one rung every three ticks. Build success never left 1.00, so no absolute floor was crossed and detection was available only through the drift modality, which fired at the injection tick itself (lag 0) on review engagement. Attribution abstained while a single rung was visible, then returned `alignment_creep` at confidence 0.73 once the pattern accumulated, selecting a Tier 1 charter reinjection. Review engagement recovered from 6.85 to 7.76 within one tick and held twelve, exiting `resolved` at a TTR of 17. A companion cell attributed the same mode correctly at 0.61 but exited `timeout`: creep has no escalation tier, so a failed Tier 1 fix has nowhere to go.

Input brittleness strips the Programmer's GUI exemplars from its role prompt, scoped to GUI-category episodes. Targeted requirement coverage collapsed from 1.00 to 0.27 while an untargeted CLI control held at 1.00, confirming that the injector degrades a category rather than the system. We report no detection-lag comparison, because the honest form of the claim is a power argument that does not need one: sample size scales as $1/\text{effect}^2$, and a category of share s dilutes its collapse to $s\times$ the effect in the aggregate, so aggregate-only monitoring needs $1/s^2$ times as many episodes—$25\times$ at our 20% share, roughly 4 episodes within the group against 104 in the aggregate. At realistic batch sizes, aggregate monitoring is not just slower to see a minority category fail; it is under-powered to see it at all. The ratio is scale-free, though the absolute counts depend on a task-mix-dependent variance estimate. Disaggregated envelopes in turn require per-group power floors (Section IV-A); in one pilot the floor correctly forced an abstention instead of a confident misattribution.

D. Attribution Quality Across All Cells

Because every injected cell carries ground truth, the funnel's output is scored as injected versus attributed mode with abstention as a scored outcome. We are not aware of a prior such accounting for a remediation protocol. Across the four cycles opened under the frozen mapping table and final contracts, three produced an attribution and all three were correct (`behavioral_drift` in τ^2; `alignment_creep` twice in ChatDev, at confidence 0.61 and 0.73); one abstained. Accuracy excluding abstentions is 3/3 at a 25% abstention rate. We report the two together, since a stage that resolves every case by construction can reach any accuracy asked of it. With $n = 4$ this is an accounting and not a confusion matrix with per-mode recall. Only APIP runs enter it: undiagnosed does not attribute, and oracle would be diagonal by fiat. The injected modes are also structurally distinct, so the funnel was never asked to separate a confusable pair. The hard case is a stale tool schema, which the precedence rule of Section III routes to drift rather than tool misuse; testing that rule against an injector that produces it remains future work.

E. Limitations

Replication is the main limitation. Each reported condition is a single seed on one injection schedule, so we report observed trajectories rather than rates or confidence intervals.

Every degradation reported here is injected, so the objection that injected drift merely mirrors its own schedule stands unanswered. Answering it requires an environment whose world evolves without injection; we discuss the one we specified for that purpose as future work (Section VI-B).

Injection schedules are scripted, and two contract metrics depend on an LLM judge. We enforce judge/backbone independence by construction and score one atomic claim at a time behind a recall-oriented pre-filter, but judge error remains a bounded unknown. Susceptibility to stale-corpus drift is itself model-dependent: policy-fact statement rate varied from 3.3% to 37.5% across the backbones we tested, so absolute thresholds do not transfer between models. Thresholds must therefore be calibrated per deployment, which is what the behavioral contract is for. Contract, mapping table, and model names are pinned in every run manifest, and all runs, traces, and the frozen mapping table are released.

A hosted backbone, however, is not a pinnable dependency. The provider deprecated our backbone on 2026-09-03, one day after the reported cells ran, and its named successor exhausted the step budget on every episode. The same weights from another vendor, on the same seed, contract, mapping table and schedule, reproduce the drift at 8-bit but not the outcome—detection at lag 2, one resolution, then three regressions and a `timeout` exit—while at 4-bit the agent stated no policy facts post-injection, so no cycle opened. The phenomenon survives the vendor change; the durable exit does not. A manifest that pins a model by provider and name records what was asked for, not what will answer.

VI. Open Problems and Future Directions

A. Multi-Agent Mesh Deployments: The New Hire Problem

The APIP as presented assumes one agent, one contract, one remediation process. In agent meshes a failure mode emerges that is structurally invisible to this assumption: *mesh disruption*. A new specialist is introduced and performs its own task well, yet peer performance declines; the APIP triggers on the peers, but attribution finds no intrinsic failure mode—no drift, fresh retrieval, intact alignment. The cause is exogenous, and a single-agent APIP cannot represent it. Kim et al. [22] report performance change against single-agent baselines from $+80.8\%$ to -70.0% depending on task structure. The phenomenon is analogous to the organizational

"new hire problem" that Event System Theory [15] frames through novelty, disruptiveness, and criticality.

We tested this and did not reproduce it. Inserting a Documentation Specialist between Programmer and Reviewer executed cleanly: it appeared in 112 of 112 post-injection episodes, produced real documentation, and its output propagated downstream. Yet no peer metric moved. Build success was 1.00 on both sides, review engagement 7.64–7.68 before versus 7.63–7.76 after, and tool error rate 0.000. No breach occurred, so no cycle opened and the onboarding arm could not be evaluated.

We read this as a precondition finding rather than a refutation. ChatDev builds each phase's prompt afresh from templates, so there is no bounded context pool to crowd, and the mechanism this environment can test, distributional shift alone, did not degrade peers. Mesh disruption therefore plausibly requires an architecture with a bounded or evicting shared context, a testable requirement on any environment used to look for it. The mesh machinery and the scoped-context onboarding protocol (quarantining a newcomer's output during a probationary window) are implemented and released; what remains is to run them in such an environment. Whether attribution stays tractable at hundreds of agents remains open.

B. Other Open Problems

Naturally occurring degradation is the most immediate next step, since every result here rests on injected failure. The environment we specified to remove that dependence is CoffeeBench [21], whose agent-operated firms inhabit a persistent economy that evolves on its own. We did not run it: its 90-day horizon resists truncation without breaking comparability, and one replication exceeded our compute budget by an order of magnitude. It is also the setting in which `recalibrated` becomes necessary, since the protocol there must separate degradation warranting intervention from legitimate environmental change.

Three further problems remain open. Contracts are architecturally foundational yet absent from most deployments, and authoring one requires prospective agreement across ML engineering, product, and legal, with no tooling to support it. Cause attribution does not yet scale, since the human adjudication step of Section IV-B needs expert time and automated step-level attribution remains weak. Ma et al. report up to 36.2% step-level attribution accuracy on Who&When [16], illustrating the difficulty of automated failure localization. Such methods attribute individual trajectories where the APIP needs window-level classification, which is the gap the funnel of Section IV-B is a first attempt to close. Organizational ownership is unresolved, since in HR the PIP is jointly owned by manager and HR function and AI deployments have no equivalent, and without an ownership model adoption will stall.

VII. Conclusion

APIP is a post-deployment remediation program that turns the human PIP's lifecycle primitives into operational functions: trigger, cause attribution, tiered intervention, bounded window, and exit evaluation. It adapts Kirkpatrick's model to distinguish durable recovery from superficial improvement, and scores exit against the contract envelope rather than the intervention log. The harness, artifacts, version pins, and frozen mapping table are released, so an auditor can check the analysis was not fitted after the fact. Inserting a competent specialist into a mesh degraded no peer, which suggests mesh disruption requires a bounded shared context. One number drives the machinery: the undiagnosed responder patched four times, regressed three times, and finished indistinguishable from taking no action, while scoring higher on per-cycle time-to-resolution. Any remediation scorecard that rewards speed without durability will prefer that responder.

Declaration on Generative AI

The conceptual framework originated with the authors and was developed further in dialogue with Claude (Anthropic; primarily Opus 5), which was also used for language editing and prose drafting throughout, to condense a longer manuscript into this format, to develop the simulation testbed that produced the reported results, and to assemble Table II. All content was reviewed and substantially revised by the authors, who take full responsibility for it. No AI system generated references.

References

[1] P. Liang *et al.*, "Holistic evaluation of language models," arXiv:2211.09110, 2022.
[2] D. Ganguli *et al.*, "Red teaming language models to reduce harms," arXiv:2209.07858, 2022.
[3] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *NeurIPS*, vol. 35, 2022.
[4] DataRobot, "The unmet AI needs survey 2026," 2026. [Online]. Available: https://www.datarobot.com/resources/unmet-ai-needs-survey-2026/
[5] M. Z. Pan *et al.*, "Measuring agents in production," arXiv:2512.04123, 2025.
[6] European Parliament and Council, "Regulation on artificial intelligence (EU AI Act)," *Official Journal of the European Union*, 2024.
[7] D. L. Kirkpatrick, *Evaluating Training Programs: The Four Levels*. San Francisco, CA, USA: Berrett-Koehler, 1994.
[8] M. Cemri, M. Z. Pan, S. Yang *et al.*, "Why do multi-agent LLM systems fail?" arXiv:2503.13657, 2025.
[9] N. V. J. K. Kartik, G. Sapra, R. Hada, and N. Pareek, "AgentCompass: Towards reliable evaluation of agentic workflows in production," arXiv:2509.14647, 2025.
[10] A. Rath, "Agent drift: Quantifying behavioral degradation in multi-agent LLM systems over extended interactions," arXiv:2601.04170, 2026.
[11] K. Zhu, X. Ye, Z. Han *et al.*, "AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents," arXiv:2607.18754, 2026.
[12] S. Verma, P. Prasun, and P. Kumar, "RAIL Guard: Closing the evaluation-to-remediation gap in responsible AI for LLM agents," arXiv:2607.16215, 2026.
[13] J. D. Kirkpatrick and W. K. Kirkpatrick, *Kirkpatrick's Four Levels of Training Evaluation*. Association for Talent Development, 2016.
[14] A. Chan *et al.*, "Visibility into AI agents," in *Proc. ACM Conf. Fairness, Accountability, and Transparency (FAccT)*, 2024.
[15] F. P. Morgeson, T. R. Mitchell, and D. Liu, "Event system theory: An event-oriented approach to the organizational sciences," *Academy of Management Review*, vol. 40, no. 4, pp. 515–537, 2015.
[16] G. Ma *et al.*, "Automatic failure attribution and critical step prediction method for multi-agent systems based on causal inference," arXiv:2509.08682, 2025.
[17] S. Zhang *et al.*, "Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, 2025, arXiv:2505.00212.
[18] Z. G. Wang, "AgentTrace: Causal graph tracing for root cause analysis in deployed multi-agent systems," arXiv:2603.14688, 2026.
[19] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan, "τ^2-bench: Evaluating conversational agents in a dual-control environment," arXiv:2506.07982, 2025.
[20] C. Qian, W. Liu, H. Liu *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.
[21] I. Sugiura *et al.*, "CoffeeBench: Benchmarking long-horizon LLM agents in heterogeneous multi-agent economies," arXiv:2606.16613, 2026.
[22] Y. Kim *et al.*, "Towards a science of scaling agent systems," arXiv:2512.08296, 2025.
[23] A. Smith, "PIPs: Write, implement and time them precisely," Society for Human Resource Management, Feb. 2023.